

Instituto Tecnológico de San Juan del Río



Tópicos de Ciberseguridad

PRÁCTICA R003 Y R004

RECONSTRUCCIÓN DEL CÓDIGO FUENTE CON GHIDRA Y COMPARACIÓN DE EJECUTABLES

P R E S E N T A:

Angel Elías Romero Paulin

Omar Israel Morales León

Esteban Reséndiz Urquiza

Ingeniería en Sistemas Computacionales

NO. DE CONTROL: 2259355, 2259353

SEPTIEMBRE 2026

1. Objetivo

Analizar un ejecutable sencillo mediante Ghidra para reconstruir en lenguaje C la lógica observada en el Decompiler. Posteriormente, recompilar el código reconstruido con Tiny C Compiler (TCC) y comparar el nuevo ejecutable con el original mediante funcionamiento, tamaño, hash SHA-256, comparación byte a byte y revisión hexadecimal con HxD.

La práctica se divide en dos reactivos: R003, enfocado en documentar el proceso de reconstrucción del código fuente; y R004, orientado a compilar el código reconstruido y comparar el ejecutable resultante con el binario original.

2. Herramientas utilizadas

- Windows y PowerShell para ejecutar comandos y validar archivos.
- Tiny C Compiler (TCC) 0.9.27 (i386 Windows) para compilar los programas en C.
- Ghidra 12.1.3 para el análisis estático y la descompilación del ejecutable.
- OpenJDK 25 como entorno requerido para ejecutar Ghidra.
- HxD para la inspección hexadecimal y comparación visual de ejecutables.
- Editor de código para revisar el archivo JSON generado y los archivos fuente en C.

3. Desarrollo de R003 - Reconstrucción del código fuente

3.1 Creación del programa original

Se creó un programa sencillo en lenguaje C cuyo propósito es sumar los números del 1 al 5 y almacenar el resultado en un archivo denominado resultado.json. El programa utiliza un ciclo for, abre el archivo con fopen, escribe el contenido mediante fprintf y finalmente lo cierra con fclose.

```
#include <stdio.h>

int main(void)
{
    int i;
    int suma = 0;
    FILE *archivo;

    for (i = 1; i <= 5; i++)
    {
        suma = suma + i;
    }

    archivo = fopen("resultado.json", "w");

    if (archivo == NULL)
    {
        return 1;
    }

    fprintf(archivo,
        "{\n"
        "  \"materia\": \"Temas de Ciberseguridad\",\n"
        "  \"valor\": %d,\n"
        "  \"estado\": \"activo\"\n"
        "}\n",
        suma
    );

    fclose(archivo);

    return 0;
}
```

Figura 1. Código fuente del programa original utilizado para generar el ejecutable de análisis.

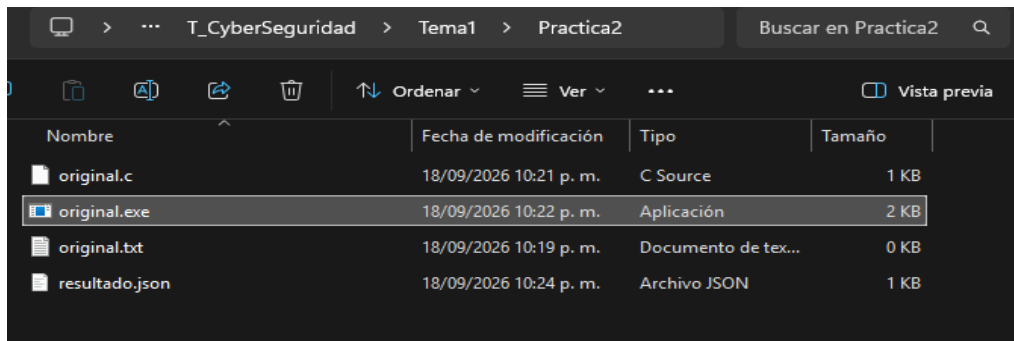
3.2 Ejecución y comprobación del programa original

El código fuente se compiló con TCC para obtener original.exe. Posteriormente se ejecutó desde PowerShell para comprobar su funcionamiento antes de realizar el análisis con Ghidra.

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\angel\OneDrive\Escritorio\T_CyberSeguridad\Tema1\Practica2> .\original.exe
```

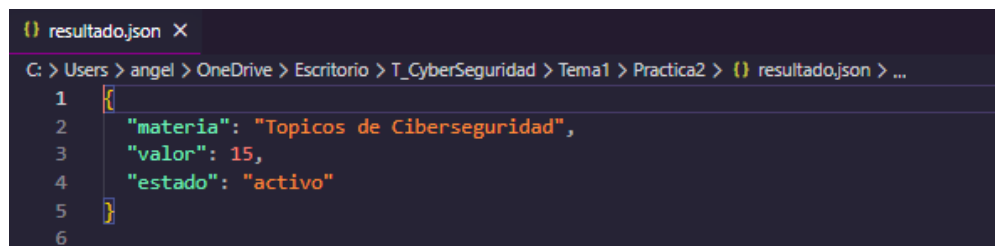
Figura 2. Ejecución de original.exe desde PowerShell.



Nombre	Fecha de modificación	Tipo	Tamaño
original.c	18/09/2026 10:21 p. m.	C Source	1 KB
original.exe	18/09/2026 10:22 p. m.	Aplicación	2 KB
original.txt	18/09/2026 10:19 p. m.	Documento de tex...	0 KB
resultado.json	18/09/2026 10:24 p. m.	Archivo JSON	1 KB

Figura 3. Archivos generados por el programa original en la carpeta de trabajo.

La ejecución creó correctamente resultado.json. El archivo contiene la materia, el valor calculado por el ciclo y un estado de prueba.



```
{
  "materia": "Temas de Ciberseguridad",
  "valor": 15,
  "estado": "activo"
}
```

Figura 4. Contenido de resultado.json generado por el ejecutable original.

3.3 Creación del proyecto e importación en Ghidra

Se creó un proyecto no compartido en Ghidra y se seleccionó original.exe como archivo de análisis. Ghidra reconoció automáticamente el binario como un ejecutable Portable Executable (PE) de 32 bits para arquitectura x86, little-endian y Windows.

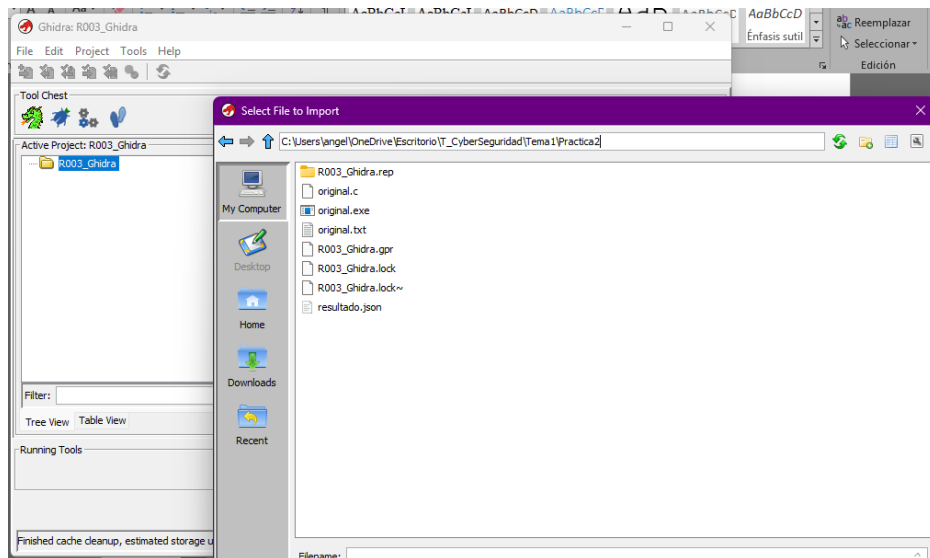


Figura 5. Selección del archivo *original.exe* para importarlo al proyecto de Ghidra.

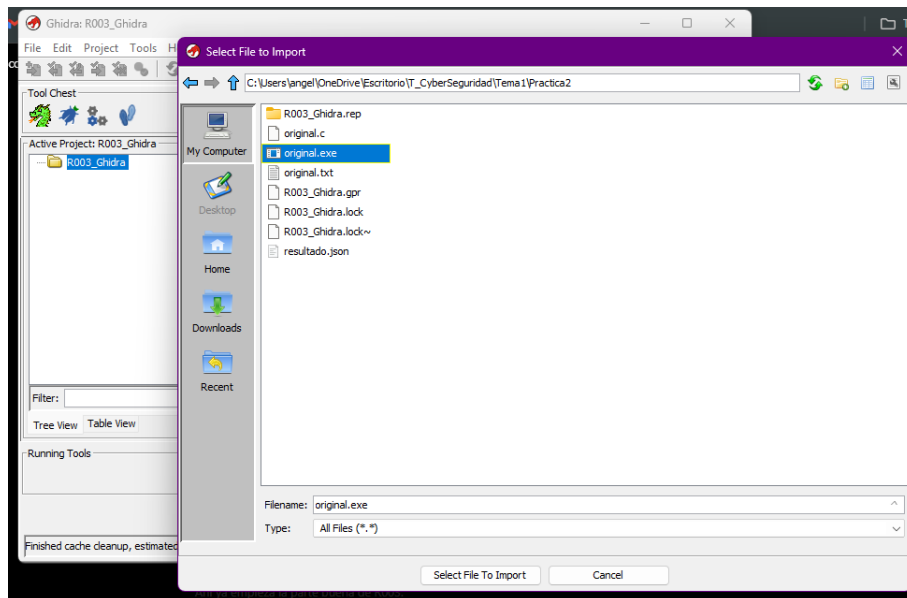


Figura 6. Selección de *original.exe* en la carpeta de trabajo.

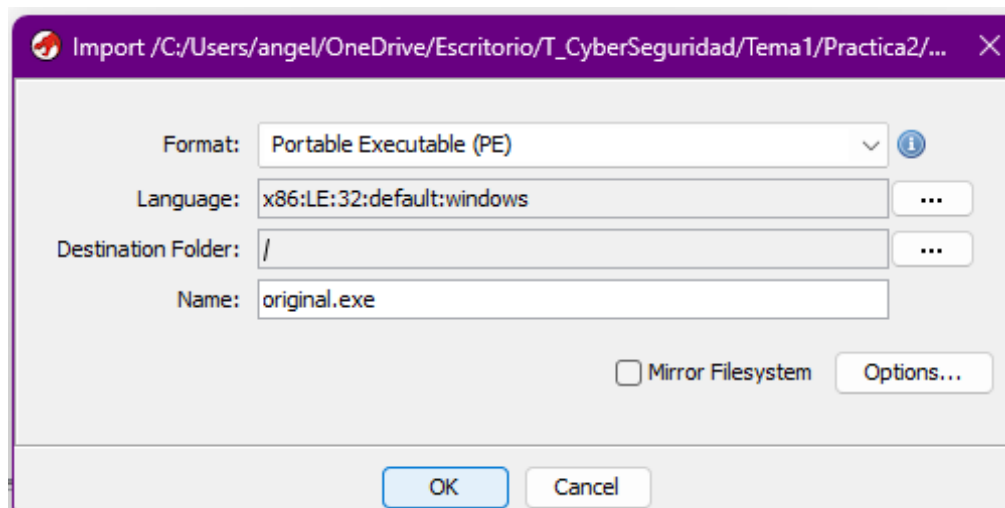


Figura 7. Parámetros detectados por Ghidra: Portable Executable (PE), x86 de 32 bits.

3.4 Análisis del ejecutable

Una vez importado el ejecutable se realizó el análisis automático con las opciones estándar de Ghidra. El resumen de importación confirmó un ejecutable de 2048 bytes y permitió continuar con la revisión del código descompilado.

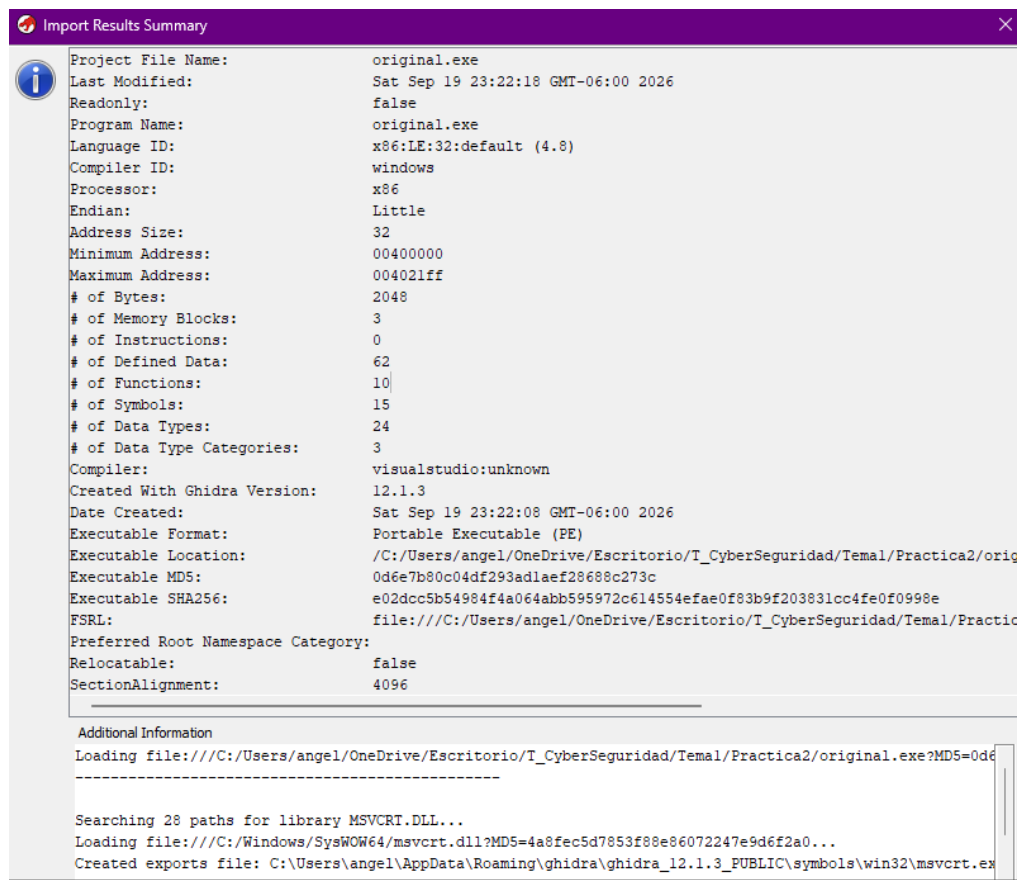


Figura 8. Resumen de importación del ejecutable original en Ghidra.

En CodeBrowser se revisó la función de entrada entry. Desde ella se identificó la llamada a FUN_00401000, función que contiene la lógica principal del programa creado para la práctica.

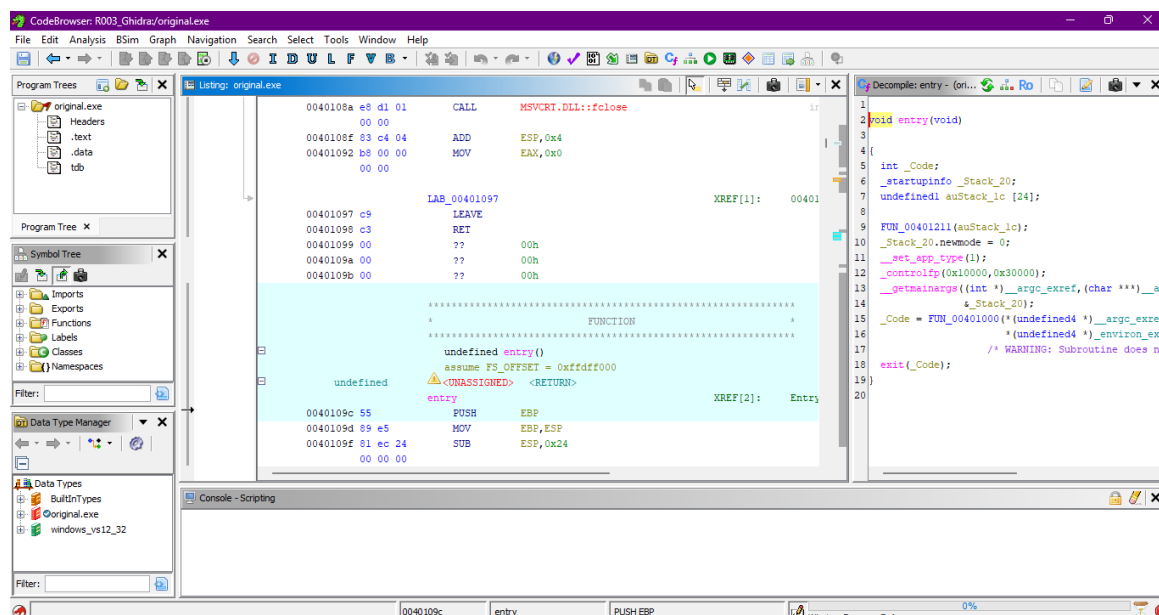


Figura 9. Vista de CodeBrowser mostrando entry y la llamada a la función principal analizada.

3.5 Reconstrucción de la lógica observada

Al abrir FUN_00401000 en el Decompiler se reconocieron tres elementos principales: un apuntador de archivo, un acumulador inicializado en cero y una variable utilizada como contador del ciclo. Para facilitar la lectura se renombraron las variables locales a archivo, suma e i.

El Decompiler mostró un ciclo equivalente a iniciar i en 1, repetir mientras i sea menor que 6 y acumular el valor en suma. Después se observan las llamadas a fopen, fprintf y fclose para crear resultado.json. Finalmente, Ghidra expresa el retorno de la función como una comparación booleana relacionada con el resultado de fopen.

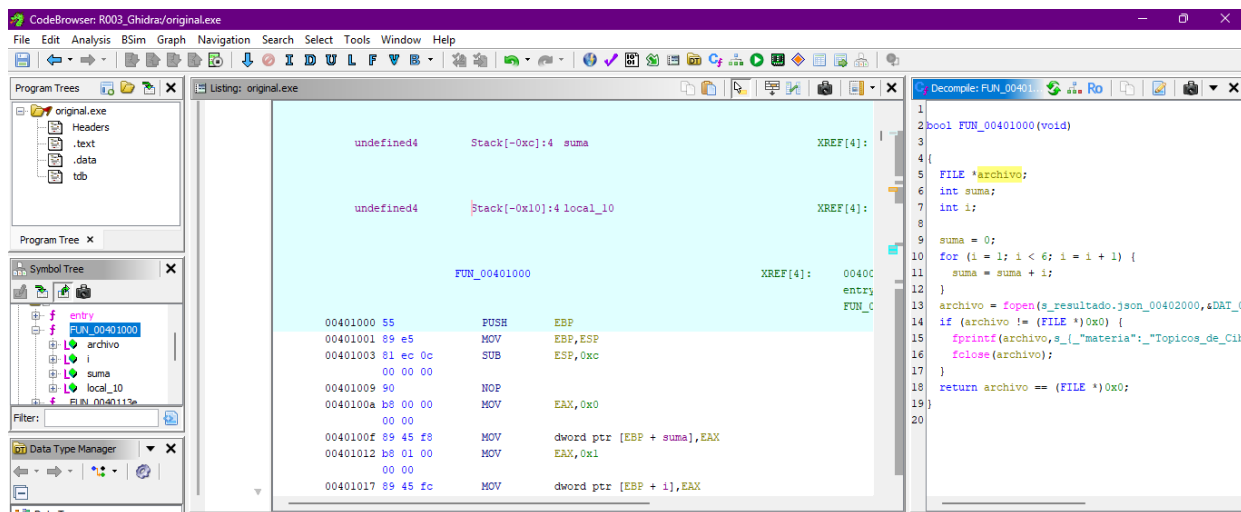


Figura 10. FUN_00401000 en el Decompiler después de interpretar y renombrar las variables principales.

A partir de esta representación se creó reconstruido.c. La reconstrucción conserva el comportamiento observado en Ghidra, aunque no busca reproducir carácter por carácter el código fuente original.

4. Desarrollo de R004 - Compilación y comparación

4.1 Compilación del código reconstruido

El archivo reconstruido.c se compiló nuevamente con TCC 0.9.27 para obtener reconstruido.exe. El ejecutable se mantuvo únicamente como material de trabajo para realizar las comparaciones solicitadas.

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\angel\OneDrive\Escritorio\T_CyberSeguridad\Temal\Practica2> .\original.exe
PS C:\Users\angel\OneDrive\Escritorio\T_CyberSeguridad\Temal\Practica2>
PS C:\Users\angel\OneDrive\Escritorio\T_CyberSeguridad\Temal\Practica2> "C:\Users\angel\Downloads\tcc-0.9.27-win32-bin\tcc\tcc.exe" reconstruido.c -o reconstruido.exe
PS C:\Users\angel\OneDrive\Escritorio\T_CyberSeguridad\Temal\Practica2> .\reconstruido.exe
PS C:\Users\angel\OneDrive\Escritorio\T_CyberSeguridad\Temal\Practica2>
```

Figura 11. Compilación y ejecución de reconstruido.exe mediante PowerShell.

Nombre	Fecha de modificación	Tipo	Tamaño
R003_Ghidra.rep	19/09/2026 11:04 p. m.	Carpeta de archivos	
original.c	18/09/2026 10:21 p. m.	C Source	1 KB
original.exe	18/09/2026 10:22 p. m.	Aplicación	2 KB
original.txt	18/09/2026 10:19 p. m.	Documento de tex...	0 KB
R003_Ghidra.gpr	19/09/2026 11:04 p. m.	Archivo GPR	0 KB
R003_Ghidra.lock	19/09/2026 11:04 p. m.	Archivo LOCK	1 KB
R003_Ghidra.lock~	19/09/2026 11:04 p. m.	Archivo LOCK~	0 KB
resultado.json	19/09/2026 11:42 p. m.	Archivo JSON	1 KB
reconstruido.c	19/09/2026 11:40 p. m.	C Source	1 KB
reconstruido.exe	19/09/2026 11:40 p. m.	Aplicación	2 KB

Figura 12. Carpeta de trabajo con original.c, reconstruido.c y los ejecutables utilizados durante la práctica.

4.2 Comprobación funcional

Se ejecutó reconstruido.exe y se verificó que generara el mismo archivo resultado.json que el programa original. El contenido obtenido mantiene el valor 15 y el estado activo, por lo que la reconstrucción conserva el comportamiento esperado.

Nombre	Fecha de modificación	Tipo	Tamaño
R003_Ghidra.rep	19/09/2026 11:04 p. m.	Carpeta de archivos	
original.c	18/09/2026 10:21 p. m.	C Source	1 KB
original.exe	18/09/2026 10:22 p. m.	Aplicación	2 KB
original.txt	18/09/2026 10:19 p. m.	Documento de tex...	0 KB
R003_Ghidra.gpr	19/09/2026 11:04 p. m.	Archivo GPR	0 KB
R003_Ghidra.lock	19/09/2026 11:04 p. m.	Archivo LOCK	1 KB
R003_Ghidra.lock~	19/09/2026 11:04 p. m.	Archivo LOCK~	0 KB
reconstruido.c	19/09/2026 11:40 p. m.	C Source	1 KB
reconstruido.exe	19/09/2026 11:40 p. m.	Aplicación	2 KB
resultado.json	19/09/2026 11:43 p. m.	Archivo JSON	1 KB

Figura 13. Archivo resultado.json generado después de ejecutar el programa reconstruido.

```

{} resultado.json x  Notas de la versión: 1.138.0 x
C: > Users > angel > OneDrive > Escritorio > T_CyberSeguridad > Tema1 > Practica2 > {} resultado.json
1  {
2    "materia": "Temas de Ciberseguridad",
3    "valor": 15,
4    "estado": "activo"
5  }
6

```

Figura 14. Contenido de resultado.json producido por reconstruido.exe.

4.3 Comparación de tamaño y SHA-256

Se compararon los tamaños de original.exe y reconstruido.exe. Ambos archivos tienen un tamaño de 2048 bytes. Posteriormente se calculó el hash SHA-256 de cada ejecutable utilizando certutil.

Archivo	Tamaño	SHA-256
original.exe	2048 bytes	e02dcc5b54984f4a064abb595972c614554efa0f83b9f203831cc4fe0f0998e
reconstruido.exe	2048 bytes	5b028496bfe12b0428b93b8483ca76e0bf55c9fafa589cdfb1274b607c78f79c

```
PS C:\Users\angel\OneDrive\Escritorio\T_CyberSeguridad\Temal\Practica2> dir original.exe reconstruido.exe

Directorio: C:\Users\angel\OneDrive\Escritorio\T_CyberSeguridad\Temal\Practica2

Mode                LastWriteTime         Length Name
----                -
-a-----         18/09/2026  10:22 p. m.         2048 original.exe

PS C:\Users\angel\OneDrive\Escritorio\T_CyberSeguridad\Temal\Practica2> certutil -hashfile original.exe SHA256
SHA256 hash de original.exe:
e02dcc5b54984f4a064abb595972c614554efa0f83b9f203831cc4fe0f0998e
CertUtil: -hashfile comando completado correctamente.
PS C:\Users\angel\OneDrive\Escritorio\T_CyberSeguridad\Temal\Practica2> certutil -hashfile reconstruido.exe SHA256
SHA256 hash de reconstruido.exe:
5b028496bfe12b0428b93b8483ca76e0bf55c9fafa589cdfb1274b607c78f79c
CertUtil: -hashfile comando completado correctamente.
PS C:\Users\angel\OneDrive\Escritorio\T_CyberSeguridad\Temal\Practica2>
```

Figura 15. Verificación del tamaño y obtención de hashes SHA-256 mediante PowerShell.

Aunque ambos ejecutables tienen el mismo tamaño, los hashes son diferentes. Esto demuestra que los archivos no son idénticos byte por byte.

4.4 Comparación byte a byte

La comparación binaria se realizó con el comando fc.exe /b original.exe reconstruido.exe. El comando reportó numerosas diferencias en distintos offsets. Algunas de las primeras diferencias encontradas fueron las siguientes:

Offset	original.exe	reconstruido.exe
0xA8	9C	98
0xD8	02	31
0xD9	B3	3E
0x180	A0	98
0x21F	05	06

Las diferencias comienzan en regiones de cabecera y continúan en la sección de código. El resultado confirma que el ejecutable reconstruido conserva la funcionalidad pero posee una representación binaria distinta.

4.5 Comparación visual con HxD

Finalmente se abrieron ambos ejecutables en HxD y se utilizó la herramienta de comparación. La revisión visual permite comprobar las diferencias detectadas previamente por fc.exe /b.

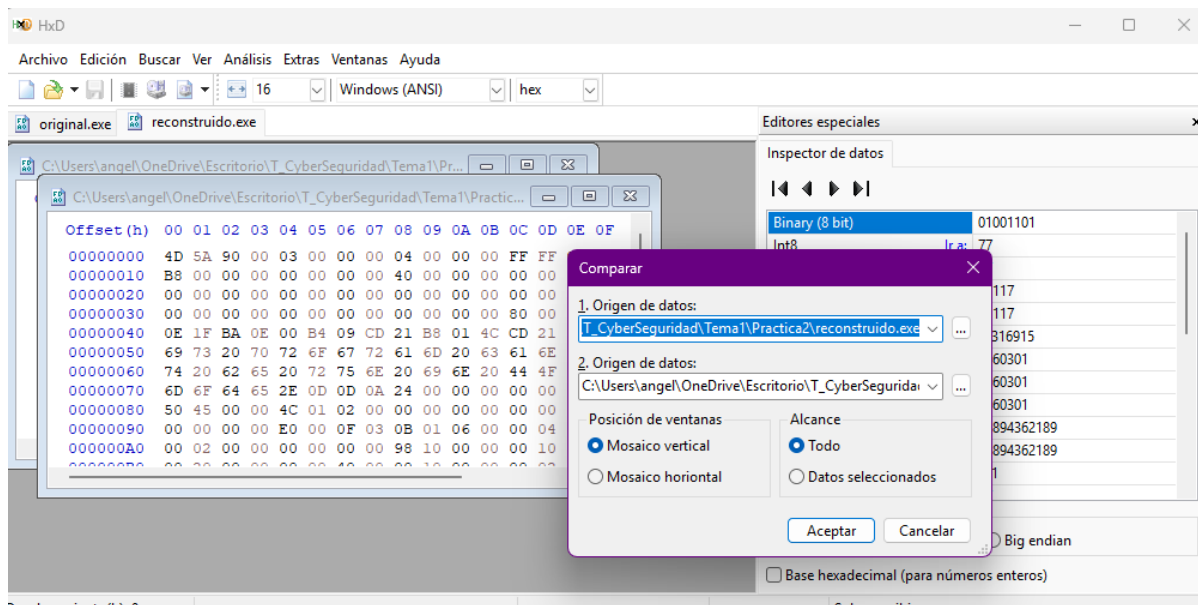


Figura 16. Configuración de la comparación entre original.exe y reconstruido.exe en HxD.

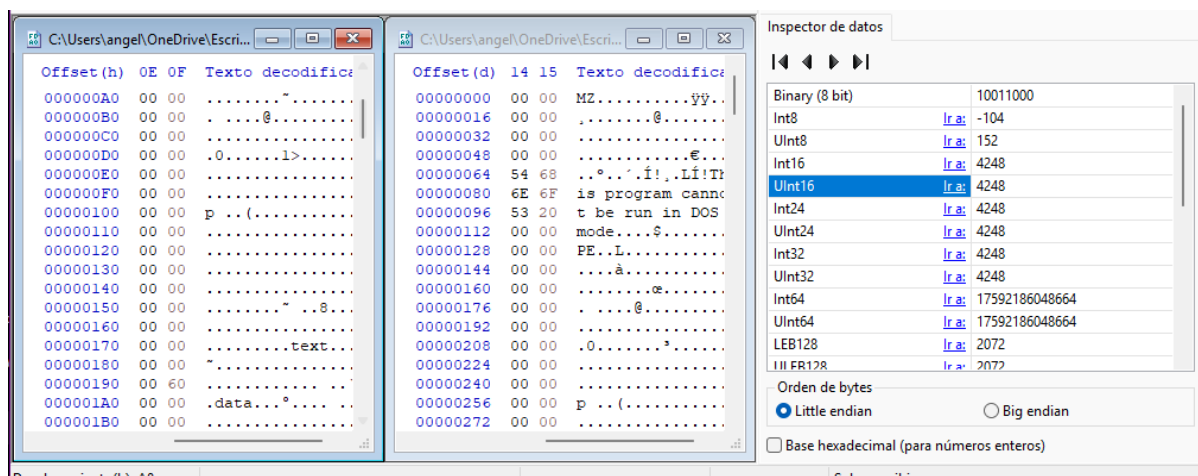


Figura 17. Comparación visual de ambos ejecutables en HxD en la zona del offset 0xA8.

En el offset 0xA8, original.exe contiene el byte 9C y reconstruido.exe contiene 98. Esta diferencia coincide con la primera variación reportada por fc.exe /b.

5. Resultados

El proceso completo permitió reconstruir un código fuente funcional a partir del análisis del ejecutable original y comprobar posteriormente la relación entre ambos binarios. Los resultados principales fueron:

- Ghidra reconoció correctamente original.exe como un ejecutable PE de 32 bits para arquitectura x86.
- Se identificó FUN_00401000 como la función que contiene la lógica principal del programa.
- El Decompiler permitió reconocer el ciclo de suma y las operaciones de escritura del archivo JSON.
- El código reconstruido fue compilado satisfactoriamente con TCC 0.9.27.
- original.exe y reconstruido.exe generan un resultado.json funcionalmente equivalente.
- Ambos ejecutables tienen un tamaño de 2048 bytes, pero sus hashes SHA-256 son diferentes.
- fc.exe /b y HxD confirmaron múltiples diferencias internas entre ambos archivos.

6. Conclusiones

La práctica permitió observar de forma directa cómo un ejecutable puede analizarse mediante ingeniería inversa para obtener una representación de alto nivel de su lógica. Ghidra facilitó la localización de la función principal y la identificación de estructuras como el ciclo for, las variables locales y las llamadas a funciones de la biblioteca estándar de C.

La reconstrucción fue suficiente para crear un segundo programa con el mismo comportamiento observable: ambos ejecutables generan un archivo JSON con el valor calculado correctamente. Sin embargo, la comparación demostró que equivalencia funcional no significa identidad binaria. Aunque original.exe y reconstruido.exe tienen el mismo tamaño, los valores SHA-256 y los bytes en distintos offsets son diferentes.

Por lo tanto, el proceso de descompilación permite recuperar la lógica esencial de un programa, pero la compilación posterior puede producir instrucciones, organización interna y metadatos distintos a los del binario de referencia.

7. Anexo - Código fuente original

```
#include <stdio.h>

int main(void)
{
    int i;
    int suma = 0;
    FILE *archivo;

    for (i = 1; i <= 5; i++)
    {
        suma = suma + i;
    }

    archivo = fopen("resultado.json", "w");

    if (archivo == NULL)
    {
        return 1;
    }

    fprintf(archivo,
        "{\n"
        "  \"materia\": \"Temas de Ciberseguridad\",\n"
        "  \"valor\": %d,\n"
```

```

        "  \"estado\": \"activo\"\n"
        "}\n",
        suma
    );

    fclose(archivo);

    return 0;
}

```

8. Anexo - Código fuente reconstruido

```

#include <stdio.h>

int main(void)
{
    FILE *archivo;
    int suma;
    int i;

    suma = 0;

    for (i = 1; i < 6; i = i + 1)
    {
        suma = suma + i;
    }

    archivo = fopen("resultado.json", "w");

    if (archivo != NULL)
    {
        fprintf(
            archivo,
            "{\n"
            "  \"materia\": \"Temas de Ciberseguridad\",\n"
            "  \"valor\": %d,\n"
            "  \"estado\": \"activo\"\n"
            "}\n",
            suma
        );

        fclose(archivo);
    }

    return archivo == NULL;
}

```